NumPy — Fast Numerical Computing in Python

ndarray | Why NumPy Beats Plain Python Lists

1. What is NumPy?

NumPy (Numerical Python) is a Python library used for fast numerical computing. It provides a powerful data structure called the **ndarray** (N-dimensional array), which stores numerical data efficiently and allows fast mathematical and scientific computations.

1.1 What NumPy is used for

NumPy supports a wide range of mathematical operations, including:

- Addition, subtraction, multiplication, division (element-wise)
- Statistics (mean, median, standard deviation, variance)
- Linear algebra (dot products, matrix multiplication, determinants)
- Matrix computations (reshaping, transposing, solving equations)

Note: An ndarray stores all of its elements as a single, fixed data type in one continuous block of memory. This is the key structural difference from a Python list, which stores a collection of separate Python objects scattered in memory — and it's the main reason NumPy is so much faster.

2. Practical Demonstration — Why NumPy is Faster

Let's measure how quickly NumPy can sum 1,000 numbers, using the same kind of timing approach we used earlier with a plain Python list.

2.1 Create a NumPy array and sum it

script.py

```python
import numpy as np
import time

# Create a NumPy array of numbers from 1 to 1000
arr = np.arange(1, 1001)

# Measure execution time
start = time.time()
print("Start:", start)

total = np.sum(arr)

end = time.time()
print("End:", end)

print("Sum:", total)
print("Time taken:", end - start, "seconds")
```

2.2 What each line does

• `np.arange(1, 1001)` — creates a NumPy ndarray containing 1, 2, 3, … 1000, similar to Python's range(), but as a single fast array object.
• `np.sum(arr)` — adds up all elements of the array using NumPy's internal, vectorized (compiled) implementation, instead of a Python-level for loop.
• The time.time() calls before and after let us measure how long the summing actually took.

Note: Running this alongside the earlier plain-Python list version (looping and adding manually) shows np.sum(arr) completing in a fraction of the time, even though both compute the exact same result.

3. Conclusion

So, instead of storing numerical data in a Python list, we use a NumPy **ndarray** because it is:

• **Faster** — operations run as compiled, vectorized code instead of Python-level loops
• **Uses less memory** — elements are packed together as a single fixed type, not individual Python objects
• **Supports mathematical operations directly** — no need to write manual loops for sums, averages, or matrix math

---

Quick Comparison: Python list vs NumPy ndarray

Aspect	Python list	NumPy ndarray
Element types	Can mix different types	Single fixed type

Memory layout	Scattered Python objects	One contiguous block
Numerical ops	Manual loops required	Built-in, vectorized
Speed (large data)	Slower	Much faster
Memory usage	Higher	Lower